

Dokumentation & Projekttagbuch

Innovation Lab 1
Jahr 2026

Projekt: **SafeCrack-Numbers**

Team: **Ereza Krasniqi**

1. Allgemeine Informationen

Projektname: SafeCrack - Numbers

Supervisor: Fabian Wagner

Innovation Lab 1, Wintersemester 2025/26

Projektteam:

Ereza Krasniqi, if23b304@technikum-wien.at

Management-Summary des Projektes

Im Projekt Safe Crack – Numbers entwickeln wir ein Unity-basiertes Rätselspiel, in dem ein digitaler Safe durch das Lösen mathematischer Zahlenreihen geöffnet wird. Der Spieler erkennt die Regel jeder Reihe selbstständig, gibt die nächste Zahl ein und muss mehrere unterschiedliche Reihen mit begrenzten Fehlversuchen korrekt lösen. Die zentralen Schwerpunkte sind eine klare Kernmechanik mit vier zufällig ausgewählten Rätseln pro Durchlauf, robuste Eingabevalidierung (inkl. leerer und ungültiger Eingaben), ein Fehlversuchs- und Reset-System sowie ein verständliches visuelles UI-Design im Safe-Stil. Ziel ist es, eine lauffähige, spielerisch konsistente Version bereitzustellen und diese reproduzierbar über einen Docker-Container auszuliefern.

Rahmenbedingungen und Projektumfeld

Das Projekt Safe Crack – Numbers wird als lokales Einzelprojekt in Unity (C#) umgesetzt und auf einem Mac-Entwicklungsrechner entwickelt. Die Anwendung ist als leicht bedienbares, kurzes Rätselspiel mit Fokus auf Usability und klarer Nutzerführung konzipiert. Technisch gelten als Rahmenbedingungen die Nutzung von Unity-Standardkomponenten (UI/TextMeshPro), nachvollziehbare Versionsverwaltung über GitHub sowie eine lauffähige Bereitstellung über einen Docker-Container der finalen Version (Web-Auslieferung). Besondere regulatorische Sicherheits- oder Normanforderungen (z. B. medizinische oder sicherheitskritische Standards) bestehen nicht; relevant sind jedoch robuste Eingabevalidierung, stabile Fehlerbehandlung und reproduzierbares Build-/Run-Verhalten. Als projektspezifische Vorgaben gelten die schrittweise, inkrementelle Umsetzung mit kleinen, nachvollziehbaren Commits, die spielbare Kernmechanik mit vier zu lösenden Zahlenreihen pro Durchlauf sowie die termingerechte Abgabe von Projektdokumentation, Präsentation, Video und containerisierter lauffähiger Version.

Semester-Roadmap

1 Person, 6 Sprints, je 2 Wochen, ca. 13h pro Sprint.

Sprint 1 (14 h) – Projektsetup & Kernidee

Unity Einarbeitung, Klarstellung der Ziele und Planung der Durchführung

Ergebnis: Überblick

Sprint 2 (12 h) – Erster Unity Entwurf, Probe mit einer Sequenz

Erstellung von erstem Entwurf, ersten Commits. Nutzung von Komponenten und erster Test.

Ergebnis: Entwurf und ersten Commits

Sprint 3 (15 h) – Fehlerfälle & Spielregeln

ungültige/leere Eingaben, Fehlversuche, Safe-Reset-Logik

Balancing (Anzahl Versuche, Rätselschwierigkeit), Rätsel Pool aufsetzen

Ergebnis: robuste spielbare Version

Sprint 4 (9 h) – UI/UX & Game-Feel

„Safe“-Design, bessere Farben/Lesbarkeit, Statusfeedback

kleine Effekte (z. B. visuelles Feedback bei richtig/falsch)

Ergebnis: präsentationsfähiger Look

Sprint 5 (17 h) – Testing, Fixes und Dokumentation

funktionale Tests, Bugfixing, Code-Aufräumung

Build-/Run-Stabilität auf Zielsystem

Ergebnis: Fertiges Projekt

Aufwandsschätzung gesamt

ca. 80 Stunden (1 Person)

Verteilung:

Entwicklung/Implementierung: ~65h

Testing/Dokumentation/Präsentation/Video/Docker: ~15 h

Collaboration & Tooling

- Versionsverwaltung: Git
- Repository/Collaboration: GitHub
- Entwicklungsumgebung: Unity 6, C#
- UI: Unity UI + TextMeshPro
- Packaging/Deployment: Docker
- Projekt-/Dokumentationsartefakte: Project Diary, Präsentation, Video

Links:

GitHub Repository: <https://github.com/erzakrsnq/SafeCrack>

Anmerkungen

/

2. Projekt-Kurzbeschreibung

Das Projekt „**Safe Crack – Numbers**“ verfolgt das Ziel, eine interaktive und technisch robuste Spielanwendung zu entwickeln, in der ein digitaler Safe durch das Lösen mathematischer Zahlenreihen geöffnet wird. Die Umsetzung erfolgt mit **Unity** und **C#** und fokussiert auf eine klar abgegrenzte Kernmechanik: Der Spielerin bzw. dem Spieler wird eine Zahlenfolge mit fehlendem Element angezeigt, deren zugrunde liegende Regel nicht explizit erklärt wird. Die Aufgabe besteht darin, diese Regel eigenständig zu erkennen und die korrekte nächste Zahl einzugeben. Damit kombiniert das Projekt spielerische Interaktion mit Mustererkennung, logischem Denken und unmittelbarem Feedback.

Im Unterschied zu rein statischen Aufgabenformaten schafft die Anwendung eine dynamische Spielsituation: Rätsel werden aus einem Pool zufällig ausgewählt, der Schwierigkeitsgrad wird über die Struktur der Folgen sowie die Anzahl zulässiger Fehlversuche gesteuert, und der Fortschritt ist für die Nutzerin bzw. den Nutzer jederzeit transparent sichtbar. Die Lösung orientiert sich damit an einem klaren Lern- und Motivationsprinzip: kurze Interaktionszyklen, eindeutige Rückmeldung und nachvollziehbare Konsequenzen bei Fehlern. Dadurch wird ein spielerischer Rahmen geschaffen, der sowohl im Demonstrationskontext als auch als prototypische Lernanwendung einsetzbar ist.

Die zentralen **Projektziele** sind:

1. Entwicklung einer lauffähigen Single-Level-Spielversion mit nachvollziehbarer Kernlogik,
2. robuste Behandlung typischer Fehlerfälle (z. B. leere oder ungültige Eingaben),
3. Implementierung eines konsistenten Versuchs- und Reset-Mechanismus,
4. verständliches, visuell stimmiges User Interface im „Safe“-Stil,
5. reproduzierbare Bereitstellung der finalen Version über einen Docker-Container.

Als **Deliverables** werden eine vollständige technische Umsetzung, ein dokumentierter Entwicklungsverlauf (Project Diary), eine editierbare Präsentationsdatei, ein Präsentationsvideo sowie eine containerisierte lauffähige Version bereitgestellt. Diese Artefakte stellen sicher, dass sowohl die inhaltliche Projektleistung als auch die technische Reproduzierbarkeit transparent nachvollzogen werden können. Insbesondere die Containerisierung dient dazu, die finale Anwendung ohne lokale Unity-Installation in einer standardisierten Laufzeitumgebung auszuführen.

Die **größten Herausforderungen** im Projekt liegen in drei Bereichen. Erstens in der inhaltlichen Balancierung der Rätsel: Die Zahlenfolgen müssen anspruchsvoll genug sein, um einen echten Denkprozess auszulösen, dürfen jedoch nicht so abstrakt werden, dass sie ohne externe Hilfen unlösbar erscheinen. Zweitens in der technischen Robustheit: Eingaben müssen zuverlässig validiert, Fehlersituationen sauber abgefangen und Spielzustände korrekt zurückgesetzt werden, ohne inkonsistente Zustände zu erzeugen. Drittens in der Usability: Das Interface muss klar strukturiert und selbsterklärend sein, damit die Nutzerin bzw. der Nutzer den Fokus auf die Rätsellogik legt und nicht durch Bedienungsprobleme abgelenkt wird.

Der **Mehrwert für Anwender*innen** liegt in einer niedrigschwelligen, interaktiven Denkanwendung mit unmittelbarem Feedback. Die Lösung trainiert Mustererkennung, fördert strukturiertes Problemlösen und zeigt, wie mathematische Logik in ein motivierendes Spielprinzip überführt werden kann. Gleichzeitig bietet das Projekt im Ausbildungskontext einen nachvollziehbaren End-to-End-Workflow von der Idee über die Implementierung bis zur reproduzierbaren Auslieferung.

Der **Scope** ist bewusst klar gehalten: umgesetzt wird ein fokussierter Prototyp mit mehreren aufeinanderfolgenden Zahlenrätseln, Versuchslimit, Fortschrittsanzeige, Zufallsauswahl aus einem Rätselpool, Erfolgs- und Fehlzuständen sowie automatischem Zurücksetzen bei Regelverletzung. **Nicht-Ziele** des Projekts sind unter anderem Multiplayer-Funktionalität, persistente Benutzerkonten, umfangreiche Backend-Anbindung, Cloud-Synchronisation oder eine vollständige Content-Erweiterung zu einem mehrstufigen kommerziellen Produkt. Diese Abgrenzung ist erforderlich, um innerhalb des vorgegebenen Zeitrahmens eine qualitativ konsistente und prüfbare Lösung liefern zu können.

Methodisch wird das Projekt inkrementell umgesetzt: Die Kernfunktionalität wird zuerst in kleinen, testbaren Schritten realisiert und danach um Fehlerbehandlung, UX-Verbesserungen und Deployment erweitert. Dieses Vorgehen reduziert Implementierungsrisiken, ermöglicht frühe Tests und erhöht die Nachvollziehbarkeit der Entwicklung über Versionskontrolle und dokumentierte Zwischenstände. Insgesamt entsteht damit eine kompakte, funktionale und formal sauber dokumentierte Lösung, die sowohl den fachlichen als auch den technischen Anforderungen des Projektformats entspricht.

3. Spezifikation der Lösung

Die Lösung wird als Unity-basierte, interaktive Einzelanwendung umgesetzt, in der die Benutzerin bzw. der Benutzer einen digitalen Safe durch das Lösen von Zahlenreihen öffnet. Die Anwendung besteht aus einer klar abgegrenzten Kernfunktionalität: Pro Durchlauf werden mehrere Zahlenfolgen angezeigt, die jeweils um die nächste logisch korrekte Zahl ergänzt werden müssen. Die Regel der Folge wird nicht explizit genannt, sondern muss aus dem Muster erkannt werden. Die Eingabe erfolgt über ein UI-Inputfeld, die Validierung über eine zentrale Spiellogik in C#.

3.1 Systemumfeld und Abgrenzung

Die Umsetzung erfolgt lokal in **Unity 6** mit **C#**. Das Projekt ist als Single-User-Prototyp konzipiert und benötigt keine externe Datenbank, kein Benutzerkonto und keine Netzwerkkommunikation zur Laufzeit.

Zum Scope gehören:

- Rätselanzeige, Eingabe, Prüfung, Fortschrittsanzeige
- Begrenzte Fehlversuche und definierte Reset-Logik
- Zufällige Auswahl von Rätseln aus einem Pool
- Erfolgszustand (Safe geöffnet) und Fehlzustand (Safe-Reset)

Nicht im Scope:

- Multiplayer-Funktionalität
- Persistente Nutzerprofile / Savegames
- Backend-Integration
- Komplexe Level-/Kampagnenstruktur über den Prototypumfang hinaus

3.2 Funktionale Anforderungen (Features)

1. **Anzeige einer Zahlenreihe mit fehlendem Wert**
Das System zeigt eine Folge mit Platzhalter (?) an.
2. **Eingabe und Validierung**
Die Benutzerin bzw. der Benutzer gibt eine ganze Zahl ein.
Leere und nicht numerische Eingaben werden abgefangen und mit Feedback beantwortet.
3. **Prüfung auf Korrektheit**
Die Eingabe wird mit der erwarteten Lösung verglichen.
Bei korrekter Eingabe wird zur nächsten Reihe gewechselt.
4. **Rätselpool mit zufälliger Auswahl**
Aus einem definierten Pool wird pro Spiel eine Teilmenge zufällig gezogen (ohne Duplikate innerhalb eines Runs).
5. **Versuchslimit über den gesamten Run**
Fehlversuche reduzieren das globale Versuchskontingent.
Bei Erreichen von 0 erfolgt ein Safe-Reset und Neustart des Durchlaufs.
6. **Spielabschlussbedingungen**
 - Erfolg: alle Zielreihen korrekt gelöst -> „Safe geöffnet“
 - Misserfolg: Versuchslimit aufgebraucht -> Reset auf Anfangszustand

3.3 Schnittstellen

- **Benutzerschnittstelle (UI):**
 - Anzeige der aktuellen Reihe
 - Status-/Feedbacktext
 - Inputfeld für Zahleneingabe
 - Button zur Prüfung
- **Deployment-Schnittstelle:**
 - WebGL-Build wird über Docker/NGINX ausgeliefert
 - Start via docker build und docker run

3.4 Qualitäts- und nicht-funktionale Anforderungen

- **Benutzbarkeit:** klare, selbsterklärende UI, eindeutige Statusmeldungen
- **Robustheit:** stabile Behandlung ungültiger Eingaben und Grenzfälle
- **Reproduzierbarkeit:** lauffähige Version per Docker unabhängig von lokaler Unity-Installation
- **Wartbarkeit:** inkrementelle Entwicklung, nachvollziehbare Commits, modularer Scriptaufbau

3.5 Technische Architektur

Die Logik ist in einer zentralen Steuerklasse (SafePuzzleUI) gebündelt:

- Verwaltung des Rätselpools und aktiver Rätsel
- Zustandsverwaltung (aktuelles Rätsel, verbleibende Versuche)
- Eingabeverarbeitung und Feedback
- Übergänge zwischen Erfolg, nächster Reihe und Reset

UI-Elemente (TMP-Text, InputField, Button) werden über Inspector-Referenzen mit der Logik verbunden.

3.6 Testkriterien / Abnahmekriterien

Die Lösung gilt als funktionsfähig, wenn folgende Kriterien erfüllt sind:

- Eine Reihe wird korrekt angezeigt und kann beantwortet werden.
- Leere oder ungültige Eingaben führen nicht zu Abstürzen und erzeugen sinnvolle Fehlermeldungen.
- Fehlversuche werden korrekt gezählt.
- Bei korrekter Eingabe erfolgt der Wechsel zur nächsten Reihe.
- Bei Erreichen des Limits erfolgt ein Reset.
- Bei erfolgreichem Abschluss aller Zielreihen wird der Safe geöffnet.
- Die WebGL-Version läuft im Docker-Container unter <http://localhost:8080>.

4. Aufwandschätzung

Schätzmethode

- Aufwand nach Arbeitspaketen aufgeteilt
- Schätzung in Teamstunden für das Semester
- Kleiner Puffer bereits enthalten

Gesamtaufwand

- **Gesamt: 75 Stunden (Team)**

Aufwand nach Arbeitspaketen

- Anforderungsanalyse & Architektur: **8 h**
- Backend-Entwicklung (API, Logik, DB): **20 h**
- Frontend-Entwicklung (UI, Nutzerfluss): **16 h**
- Security/Authentifizierung: **8 h**
- Tests & Qualitätssicherung: **10 h**
- Deployment/Setup (inkl. Nginx): **6 h**
- Dokumentation & Präsentation: **7 h**
- **Summe: 75 h**

Risikotreiber

- Integrationsaufwand zwischen Frontend/Backend
- Unerwartete Bugs in Authentifizierung oder Konfiguration
- Zeitverlust durch Fehlersuche kurz vor Abgabe

Pufferstrategie

- Priorisierung von Must-have-Funktionen
- Frühe Integrationstests
- Notfallscope für Nice-to-have reduzieren

5. Auslieferung

Lieferumfang

- Vollständiger Source-Code (Frontend, Backend, Konfiguration)
- Dokumentation (Setup, Betrieb, Troubleshooting)
- Beispiel-Konfigurationen inkl. nginx.conf
- Übergabe einer lauffähigen Demo-/Produktivversion

Systemarchitektur & Datenhaltung

- Client/Frontend kommuniziert über API mit Backend
- Reverse Proxy über Nginx (Routing, TLS-Termination)
- Persistenz in Datenbank (Benutzerdaten, Logs, Statusdaten)
- Trennung von App- und Infrastruktur-Konfiguration per .env

Lizenzen & Drittsoftware

- Verwendete Frameworks und Libraries inkl. Lizenztyp dokumentiert
- Keine unklaren/inkompatiblen Lizenzen im Produktivstand
- Copyright- und Attribution-Hinweise im Repository

Hardware-/Betriebsvorgaben

- Server mit Docker/Container-Runtime oder Linux-VM
- Mindestanforderungen: 2 vCPU, 4 GB RAM, stabile Netzwerkverbindung
- Offene Ports für Webzugriff (z. B. 80/443)

Installation & Inbetriebnahme

- Repository klonen, Umgebungsvariablen setzen, Abhängigkeiten installieren
- Services starten (z. B. per Docker Compose oder Startskripten)
- Nginx aktivieren, Domain/TLS konfigurieren
- Health-Check und Smoke-Test durchführen

Migration & Übergabe

- Initiale Datenbankmigrationen ausführen
- Backup-/Restore-Prozess dokumentiert
- Betriebsübergabe inkl. Monitoring- und Incident-Checkliste
- Verantwortlichkeiten (Dev/Ops) klar definiert

6. Unser Projekt-Tagebuch

Der Verlauf von SafeCrack war insgesamt sehr positiv. Ich habe das Projekt eigenständig geplant, umgesetzt und Schritt für Schritt weiterentwickelt. Durch eine klare Struktur bei Aufgaben, Prioritäten und Meilensteinen konnte ich kontinuierlich Fortschritte machen und den Überblick über alle Projektbereiche behalten.

In den ersten Phasen lag der Fokus auf der technischen Grundlage: Architektur, Grundfunktionen und stabile Entwicklungsabläufe. Danach habe ich die einzelnen Komponenten systematisch erweitert und regelmäßig getestet, damit neue Änderungen nicht zu unerwarteten Nebenwirkungen führen. Diese Vorgehensweise hat sich bewährt, weil ich dadurch Probleme früh erkennen und direkt beheben konnte.

Der größte Stolperstein im gesamten Projekt war der Bereich **Web-Build/Deployment**. Hier traten wiederholt Schwierigkeiten auf, insbesondere bei der Build-Konfiguration und bei der zuverlässigen Bereitstellung der Web-Version. Diese Probleme waren zeitintensiv und haben mehr Aufwand verursacht als ursprünglich eingeplant. Ich habe die Ursache schrittweise eingegrenzt, verschiedene Konfigurationsvarianten ausprobiert und mehrere Build- sowie Testdurchläufe durchgeführt, bis die Web-Auslieferung stabil funktioniert hat.

Rückblickend war gerade diese Phase sehr lehrreich: Ich habe ein deutlich besseres Verständnis dafür entwickelt, wie wichtig saubere Build-Prozesse, reproduzierbare Abläufe und eine klare Trennung von Entwicklungs- und Deployment-Konfiguration sind. Außerdem habe ich gelernt, bei komplexen Fehlerbildern strukturiert vorzugehen und Änderungen nachvollziehbar zu dokumentieren.

Fazit: Das Projekt lief insgesamt gut und zielgerichtet, mit einer wesentlichen Herausforderung im Web-Build-Bereich. Diese konnte ich eigenständig lösen, sodass SafeCrack am Ende in einer stabilen und präsentierbaren Form ausgeliefert werden konnte.